DAY 5 · STUDY GUIDE

Phase 5
Video Ingestion

OpenCV frame sampling · Tesseract on frames · Near-duplicate skip · Timestamp metadata · The shared seam · A real Pydantic bug

> **Today's win in one line:** The engine now ingests video, not just PDFs. A single `POST /ingest/video` samples frames every couple of seconds with OpenCV, OCRs each with Tesseract, drops near-duplicate slides, and stores the text as chunks tagged with `timestamp_seconds` and `frame_number` — so a retrieved answer can cite "0:04 in the video" the same way a PDF answer cites "page 3." Proven end-to-end with a synthetic slideshow built from the Day 1 guide: 10 slides in, 10 deduped chunks out, retrieved back with `timestamp_seconds: 4` . And a real Pydantic response-model bug got found and fixed along the way.

What you built / verified	How
Video tooling installed	`opencv-python` (frame reading), `imageio` + `imageio-ffmpeg` (bundled encoder, no separate Windows install).
Synthetic slideshow generator	`tests/make_slideshow_video.py` turns any PDF into a slides MP4. Same "known-content test input" trick as Phase 4's synthetic scan.
Video parser (`app/parsers/video_parser.py`)	OpenCV samples 1 frame / 2s, OCRs each, drops blanks (<15 chars) and near-duplicate slides (≥90% text similarity). Returns `(timestamp, frame_number, text)` .
Qdrant store extended	`upsert_chunks` accepts optional `timestamps` / `frame_numbers` ; payloads carry them for video. Search returns them. PDF path untouched.
Ingestion service extended	New `ingest_video()` + `VideoIngestionReport` , reusing the existing `_ingest_texts` embed+store seam.
Video endpoint	`POST /ingest/video` — saves upload to disk (OpenCV needs a path), 200 MB cap, ingests.
Bug found & fixed	`/search` Pydantic `SearchHit` model was silently dropping the new timestamp fields. Added them to the model.

What you built / verified	How
End-to-end verified	Slideshow ingested (10 frames, 10 chunks), retrieved with `timestamp_seconds: 4`, `frame_number: 40`, score 0.71.

Companion docs: AI Ingestion Engine Build Plan · Day 1 · Day 2 · Day 3 · Day 4 Study Guides

1 Video is just images over time

1.1 The core insight that made this cheap to build

You did not build a new pipeline today. You built a new *extractor* and reused everything downstream. A video is a sequence of still images (frames) played fast. Each frame is exactly the same problem as a scanned PDF page from Phase 4: pixels that need OCR. So once you can turn a video into "a list of frames with text," it flows through the identical chunk → embed → store pipeline you already wrote.

DEFINITION Video ingestion = sample frames, OCR each, attach time metadata, then hand off to the existing shared pipeline.

WHERE USED Recorded lectures, webinars, screen recordings, slide decks presented on camera, training videos — anywhere the useful information is on-screen text.

EXAMPLE Your slideshow MP4: 10 slides → sampled frames → Tesseract text → chunks in Qdrant, each tagged with the second it appeared.

KEY TERMS Frame, frame sampling, FPS, OCR, timestamp metadata.

WHY FOR INTERNSHIP "Multimodal" sounds hard. The honest answer is that it's the same engine with a different front door — and being able to say that clearly shows you understand your own architecture.

1.2 The seam that made it one-line-clean

Back in Phase 1, past-you wrote a helper called `_ingest_texts` and left a comment: "Phase 5's video extractor will feed into this same helper, so the engine is only written once." Today that paid off. `ingest_video` does its own extraction, then calls the exact same `_ingest_texts` that `ingest_pdf` calls.

> **YOU PROVED THIS TODAY**
>
> The only thing you had to add to the shared seam was two optional parameters (`timestamps`, `frame_numbers`). The embed-and-store logic itself didn't change at all. That's what a good abstraction boundary buys you.

2 OpenCV frame sampling

2.1 What OpenCV does here

OpenCV (imported as `cv2`) is a computer-vision library. For this project it does one job: open a video file and hand you frames one at a time as image arrays. You don't process every frame — that would be wasteful and slow. You sample.

DEFINITION OpenCV is a library for reading, writing, and processing images and video. `VideoCapture` is its video reader.

WHERE USED Your `video_parser.py` — `cv2.VideoCapture(path)` opens the file, `cap.read()` pulls the next frame, `cap.get(cv2.CAP_PROP_FPS)` reads the frame rate.

EXAMPLE At 10 FPS, sampling every 2 seconds means keeping 1 frame out of every 20. A 30-second video = 300 frames = ~15 sampled.

KEY TERMS `VideoCapture`, `read()`, FPS, frame interval, BGR color order.

WHY FOR INTERNSHIP This is the roadmap's explicit "develop applications with OpenCV" requirement, satisfied in a real pipeline rather than a toy demo.

2.2 Why sampling, and how the interval is chosen

Processing every frame is pointless for slide-style content — nothing changes 30 times a second. You pick an interval that's frequent enough to catch every distinct on-screen state but sparse enough to keep OCR time reasonable. You used **every 2 seconds**.

Strategy	Result
Every frame	Correct but absurdly slow — OCR runs hundreds of times per minute of video.
Every 2 seconds (yours)	Catches every slide, keeps OCR count low. A 30s video needs ~15 OCR passes.
Scene-change detection (advanced)	Best of both — OCR only when the picture actually changes. A future upgrade.

> **TRAP**
> OpenCV returns frames in **BGR** color order (blue-green-red), not RGB. Tesseract and PIL expect RGB. You convert with `cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)` before OCR. Skip that and colors are swapped — usually still OCR-able for text, but wrong for any color-dependent work.

3 The near-duplicate skip

3.1 The problem: one slide, many identical frames

A slide held on screen for 3 seconds produces multiple sampled frames with identical text. Without de-duplication, that's 2-3 identical chunks in Qdrant per slide — wasted storage, and duplicate hits polluting search results.

DEFINITION	Skipping a sampled frame whose OCR text is nearly identical to the previous kept frame's text.
WHERE USED	Inside the sampling loop, right after OCR, before creating a chunk.
EXAMPLE	Slide 3 appears at 0:06 and 0:08 (two samples). The second is 100% identical text, so it's skipped. Only one chunk for slide 3.
KEY TERMS	De-duplication, similarity ratio, `SequenceMatcher`, threshold.
WHY FOR INTERNSHIP	Shows you thought about data quality, not just "make it run." Duplicate chunks are a classic RAG smell.

3.2 How the similarity check works

You use Python's built-in `difflib.SequenceMatcher` to compare the current frame's text to the last kept frame's text. It returns a ratio from 0 (nothing in common) to 1 (identical). If that ratio is ≥ **0.90** (90% similar), the frame is treated as the same slide and skipped.

> **YOU PROVED THIS TODAY**
>
> Your 10-slide video, each held for 3 seconds, produced exactly **10 chunks** — one per slide. If dedup were broken you'd have seen 20-30. Landing on a clean 10 is the visible signature of the skip working.

> **WHY 0.90 AND NOT 1.0**
>
> Exact-match (1.0) would fail to collapse frames where OCR made a tiny error on one pass but not the other — same slide, slightly different text. 0.90 tolerates OCR noise while still treating genuinely different slides as distinct.

4 Timestamp metadata

4.1 Why video chunks need different metadata than PDFs

A PDF citation is a page number. A video citation is a *time*. So video chunks store `timestamp_seconds` and `frame_number` where PDF chunks store `page_number`. This is the entire point of multimodal RAG: an answer you can trace back to its exact source, whatever the medium.

DEFINITION	`timestamp_seconds` = the whole-second offset where the frame appears. `frame_number` = the absolute frame index (for tracing the exact still).
WHERE USED	Computed in the parser as `frame_index / fps`, stored in the Qdrant payload, returned by search.
EXAMPLE	Your retrieved chunk came back with `timestamp_seconds: 4, frame_number: 40` — "this text appeared at 0:04, frame 40."
KEY TERMS	Metadata, payload, provenance, source citation, multimodal.
WHY FOR INTERNSHIP	"Answer with a timestamp you can jump to" is the demo that makes video RAG feel real to a non-technical stakeholder.

4.2 How it threads through without touching the PDF path

You extended `upsert_chunks` with two *optional* parameters that default to `None`. When present (video), the fields get added to the payload. When absent (PDF), nothing changes. Same for the shared `_ingest_texts` seam. This is additive design — new capability, zero risk to the working PDF path.

> **THE PRINCIPLE**
>
> When extending a shared function, optional-with-safe-default parameters let you add a feature for one caller without forcing every other caller to change. The PDF path never even knows the video fields exist.

5 The Pydantic bug you caught

5.1 The symptom

After ingesting the video, search results came back *without* the timestamp field — not `null`, but completely absent. The data was clearly being written (the parser computed it, the store payload included it), yet it never reached the client.

THE CLUE Absent, not null. A stored-but-not-returned value would show as `null`. A completely missing field means something is *filtering* it out downstream.

THE METHOD You grepped every file for `page_number` to find where the response was shaped — because timestamp needs to live wherever page_number already does.

5.2 The cause and the fix

The `/search` endpoint returns a Pydantic model, `SearchHit`, written back in Phase 2. It declared `page_number` but nothing about timestamps. When the route did `SearchHit(**h)`, Pydantic v2 **silently ignores keys not declared in the model** — so `timestamp_seconds` and `frame_number` were dropped at the API boundary. The fix: add those two fields to `SearchHit`. One file, three lines.

> **YOU PROVED THIS TODAY**
>
> You isolated a data-flow bug by reasoning about *where a value disappears* — parser has it, store has it, search returns it, model drops it. That diagnostic path (absent vs null → grep for the sibling field → find the response model) is exactly how you'd debug this in production.

> **TRAP**
>
> Pydantic silently dropping undeclared fields is a double-edged sword. It protects you from leaking internal fields, but it also means adding data to a payload isn't enough — you must also add it to every response model in the path. Grep for a sibling field whenever a new field mysteriously vanishes.

6 The duplicate-ingest finding

6.1 Re-ingesting doesn't replace — it duplicates

While debugging, you re-ingested the video and noticed Qdrant now had it *twice*. The cause: every ingest generates a fresh random `file_id` via `uuid4()`, which seeds fresh deterministic point IDs, so the second run inserted a whole new copy instead of updating the first.

DEFINITION	Because point IDs derive from a per-run random `file_id`, the same file ingested twice becomes two independent sets of points.
THE TEMPORARY FIX	A one-off `clear_video_chunks.py` that deletes all `source_type == "video"` points, so you could re-ingest one clean copy.
THE REAL FIX (PHASE 7)	Content-hash-based file IDs — hash the file bytes so the same file always maps to the same ID, making re-ingest idempotent.

> **INTERVIEW FRAMING**
>
> "I found during testing that re-ingesting duplicates content, because file_id is random per run. The fix is content-hash file IDs, which I've scoped for the polish phase." Naming a real limitation and its fix is more convincing than claiming everything is perfect.

7 How you tested — the synthetic slideshow

7.1 The test artifact

Same philosophy as Phase 4: no real video needed. You wrote a helper that renders each PDF page as a frame, holds it for 3 seconds, and encodes an MP4. Known content, guaranteed-OCR-able text, reproducible by anyone who clones the repo.

DEFINITION	A generator that turns a PDF into a slideshow video for controlled testing.
WHERE USED	Tests only. `python tests/make_slideshow_video.py Day1_Study_Guide.pdf Day1_SLIDESHOW.mp4`
EXAMPLE	10 pages → 10 slides → 30-second MP4 at 10 FPS.
KEY TERMS	Test fixture, synthetic data, reproducible test, ffmpeg encoding.
WHY FOR INTERNSHIP	"I built a three-modality benchmark from one document — text PDF, scanned PDF, and video — and each returns comparable retrieval scores" is a genuinely strong story.

7.2 The even-dimensions gotcha

Your first encode failed: `height not divisible by 2 (828x1171)`. The H.264 codec requires even pixel dimensions. Your PDF rendered to an odd height. Fix: crop one pixel off any odd side before writing the frame — invisible to the eye, keeps the encoder happy.

> **TRAP**
>
> Almost every real-world video has even dimensions by luck, so this bug is rare and surprising. When you generate frames programmatically, always force even width and height before H.264 encoding.

7.3 The findings

Ingest response for the slideshow:

- `frames_ingested: 10` — dedup collapsed each slide to one
- `chunks_created: 10`
- `source_type: "video"`

Search response (filtered to `source_type: "video"`):

- Score **0.71** — video-frame OCR embeds as well as scanned-PDF OCR did
- `timestamp_seconds: 4`, `frame_number: 40` — the citation metadata, intact

> **YOU PROVED THIS TODAY**
>
> Same Day 1 content now flows through three ingestion paths — native text, OCR-on-scanned-PDF, and OCR-on-video-frame — and all three retrieve at comparable scores. One document became a full multimodal benchmark of your own engine.

Q&A Likely interview questions on today's work

Answers in your voice — the way you'd actually explain it, not a textbook.

Question	Your answer
How does your video ingestion work at a high level?	A video is just images over time. I open it with OpenCV, sample one frame every 2 seconds, run Tesseract OCR on each frame, drop blanks and near-duplicate slides, then push the text through the exact same chunk-embed-store pipeline my PDFs use. The only difference is metadata — video chunks carry a timestamp and frame number instead of a page number.

Question	Your answer
Why didn't you build a separate pipeline for video?	Because I didn't need to. Each video frame is the same problem as a scanned PDF page — pixels that need OCR. Back in Phase 1 I'd built a shared "embed and store" helper on purpose, so video ingestion just does its own frame extraction and then calls that same helper. I only had to add two optional metadata parameters. The core pipeline is written once.
Why sample frames instead of processing all of them?	For slide content nothing changes 30 times a second, so OCRing every frame is pure waste. I sample every 2 seconds — frequent enough to catch every slide, sparse enough that a 30-second video only needs about 15 OCR passes instead of 300.
How do you avoid duplicate chunks from a slide that's on screen for a while?	After OCRing each sampled frame I compare its text to the last frame I kept, using Python's SequenceMatcher, which gives a similarity ratio. If it's 90% or more similar I treat it as the same slide and skip it. That's why my 10-slide video produced exactly 10 chunks instead of 20 or 30.
Why 90% similarity and not exact match?	Because OCR isn't perfectly consistent — the same slide sampled twice might come out with one tiny character difference. Exact match would fail to collapse those. 90% tolerates OCR noise while still keeping genuinely different slides separate.
What metadata do video chunks store?	timestamp_seconds and frame_number, instead of the page_number that PDF chunks use. The timestamp is the frame index divided by the video's FPS. That lets a retrieved answer cite "found at 0:04 in the video" — the video equivalent of citing a page.
You hit a bug where timestamps weren't showing up. Walk me through it.	After ingesting, search results came back with no timestamp field at all — not null, completely absent. That absence was the clue: a stored-but-not-returned value would show as null, so something was filtering it out. I grepped for page_number to find where the response gets shaped, and found the /search Pydantic model, SearchHit. It declared page_number but not timestamp. Pydantic silently ignores keys you don't declare, so it was dropping my fields at the API boundary. I added the two fields to the model and they flowed through.
What did that bug teach you about Pydantic?	That adding data to a payload isn't enough — you also have to declare it in every response model along the path, or Pydantic quietly drops it. It's protective, it stops you leaking internal fields, but it also means a new field can vanish silently. My rule now: if a new field disappears, grep for a field that's already working and put mine right next to it.

Question	Your answer
You mentioned re-ingesting creates duplicates. Why?	Every ingest generates a fresh random file_id, and my Qdrant point IDs are derived from that file_id. So the same file ingested twice gets two different sets of point IDs — a second copy instead of an update. The proper fix is content-hash file IDs, so identical bytes always map to the same ID and re-ingest becomes idempotent. I've scoped that for the polish phase.
How did you test without a real video?	I generated one. A helper renders each page of my Day 1 guide as a slide, holds it three seconds, and encodes an MP4. Known content, clean text, and anyone who clones the repo can regenerate it. It also gave me a nice property: the same document now runs through text, scanned-PDF, and video paths, so I can compare retrieval quality across all three.
Your first encode failed on dimensions — what happened?	H.264 requires even pixel width and height. My PDF rendered to 828 by 1171 — the height's odd — and libx264 refused to encode. I crop one pixel off any odd side before writing the frame. Invisible to the eye, keeps the encoder happy. Most real videos are even by luck, so it's a rare surprise.
Why does OpenCV need color conversion before OCR?	OpenCV loads frames in BGR order — blue, green, red — but Tesseract and PIL expect RGB. So I convert with cvtColor and the BGR2RGB flag before OCR. For plain text it'd often still read okay, but it's wrong and I don't want color-swapped data if I ever add color-dependent processing.
How good is the OCR quality on video frames?	Good. My retrieved video chunk scored 0.71 similarity against the query — right in line with the 0.65 I got from scanned-PDF OCR in Phase 4. So going through pixels and Tesseract instead of a text layer costs only a small amount of semantic precision. It's competitive with direct extraction.
Why save the video to disk instead of processing it in memory?	OpenCV's VideoCapture reads from a file path, not from raw bytes in memory the way my PDF parser does. So the video endpoint writes the upload to storage/uploads first, then hands OpenCV that path. It's a small difference in the route, but the ingestion logic downstream is identical to PDF.
Walk me through the full / ingest/video request.	The route checks the extension and a 200 MB size cap, then writes the upload to disk because OpenCV needs a path. The service opens it, reads FPS, and samples one frame every two seconds. Each sampled frame gets converted BGR to RGB, OCR'd by Tesseract, and screened — blanks under 15 characters dropped, frames 90% similar to the last kept one skipped. Surviving frames become chunks tagged with timestamp and frame number, which go through the same embed-and-store seam as PDFs. The response reports how many frames and chunks were ingested.

Day 5 · Phase 5 complete. Video ingestion working end-to-end — OpenCV frame sampling, Tesseract OCR, near-duplicate skip, timestamp-cited retrieval. The ingestion engine is now feature-complete across text PDFs, scanned PDFs, and video. Remaining: Phase 6 (n8n workflow automation), Phase 7 (polish — logging, content-hash dedup, README, demo). Roadmap items still open: OpenCV/DeepFace face app, ReAct agent, standalone n8n workflows.